DDCA PROJECT · BATCH 2

MODULE 3

Design of a Four-Bit Binary to Decimal Conversion

Module 3 Basics: Sequential Logic — Giving the Machine Memory

A quick look at how registers, counters and flip-flops — the memory elements our converter itself doesn't need — can still be used around it to hold its input and test it automatically.

OVERVIEW

What This Project Uses From Module 3

1 Registers

A row of flip-flops that could hold our 4-bit input steady before decoding.

2 Counters

A simple way to automatically cycle through all 16 inputs to test our decoder.

3 Clock & timing

The idea of a clock edge, so stored values change at a known, reliable moment.

4 FSMs (just the idea)

Our decoder itself doesn't need one — but we see where it would fit in a bigger system.

AN IMPORTANT DISTINCTION

Our Converter Is Combinational — Here's the Contrast

COMBINATIONAL (our converter)

- Output depends only on the current input
- No clock, no memory, no “state”
- Built from gates: NOT, AND, OR …

SEQUENTIAL (this module)

- Output depends on current input AND what was stored before
- Needs a clock to know exactly when to update
- Built from latches, flip-flops and registers

BASIC MEMORY ELEMENTS

Latches & Flip-Flops: The Basics

SR Latch

The simplest memory cell — Set makes it 1, Reset makes it 0, and it holds that value otherwise.

D Latch

Stores whatever is on its D input, but only while a control signal is “open” (level-sensitive).

D Flip-Flop

Stores the D input at the exact instant the clock ticks (edge-triggered) — the building block of registers.

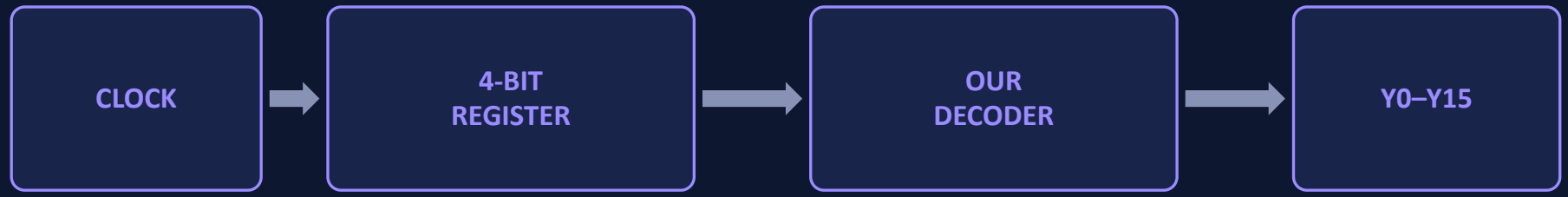
Level-sensitive (latch) reacts whenever the control signal is high; edge-triggered (flip-flop) reacts only at the instant the clock changes — flip-flops are what real registers use.

The Clock & Synchronous Design

- A clock signal ticks at a steady rate, so every flip-flop in a circuit updates at the same, predictable moment.
- This “synchronous” design is what makes large digital systems reliable — everything changes in step, rather than drifting out of sync.
- Setup and hold time (just the intuition): each flip-flop's input needs to be stable for a brief moment just before and just after the clock edge, or its stored value can't be trusted.

Registers: Holding Our 4-Bit Input

- A register is just 4 D flip-flops side by side, one per bit — enough to store one 4-bit value at a time.
- Our decoder itself has no memory, but in a real system its input would usually come from a register, so the value stays steady while the decoder reads it.



clock tells the register exactly when to latch a new value

A CLOSE RELATIVE

Shift Registers (Brief)

- A shift register is a chain of flip-flops where, on every clock tick, each bit moves into its neighbour — the whole value shifts left or right by one position.
- It's useful for turning a stream of single bits into a parallel value (or back again) — for example, receiving a 4-bit number one bit at a time over a single wire, then reading all 4 bits at once.
- Our project doesn't need one: the 4-bit input to our decoder already arrives as 4 bits side by side, so there's nothing to shift in or out.

A PERFECT USE CASE

Counters: Auto-Testing Our Decoder

- A synchronous counter is a register that adds 1 to itself on every clock tick — the same pattern used for a CPU's program counter, which tracks “which instruction is next.”
- For our project, a 4-bit counter is a natural testbench: let it count 0000 → 1111 automatically, feed each value into our decoder, and watch Y0 through Y15 light up in order.

CLOCK → 4-bit counter: 0000, 0001, 0010, …, 1111 → our decoder → Y0, Y1, Y2, …, Y15

SCOPE CHECK

Finite State Machines — Not Needed Here

- An FSM describes step-by-step behaviour using states and transitions — it's the technique behind a control unit.
- Our decoder has no steps to follow: it just reads 4 bits and lights one output, all in one go — there's no “next state” to transition to.
- FSMs matter for this module because a real control unit — deciding what a CPU does next, instruction by instruction — is exactly this kind of step-by-step machine.

FROM SPEC TO STATES

Designing a Small FSM: A Test Controller

A worked example — an FSM that could drive the counter testbench from Slide 8.

IDLE

Waiting for a start signal. Counter is held at 0000.

RUN

Counter increments every clock tick, feeding our decoder.

DONE

Counter reached 1111; all 16 outputs have been tested.

Specification → states and transitions → flip-flops to hold the current state → logic to decide the next state. This is the exact same disciplined flow used for a real CPU control unit.

FROM DIAGRAM TO TABLE

The Test Controller as a State Table

The same IDLE → RUN → DONE machine, written the way an FSM is usually implemented

Current State	Input / Condition	Next State	Action
IDLE	start = 1	**RUN**	RegWrite counter ← 0000
IDLE	start = 0	**IDLE**	hold — no change
RUN	count < 1111	**RUN**	counter ← counter + 1
RUN	count = 1111	**DONE**	stop incrementing
DONE	—	**DONE**	hold — test finished

A state table is just the state diagram's arrows written as rows — both describe the exact same machine; the table is what you'd hand to a flip-flop-and-gate implementation.

MODULE DELIVERABLE

Milestone: Register, Counter & a Small FSM

Register

A 4-bit register built and tested in the simulator, holding one steady value.

Counter

A synchronous counter cycling 0000 through 1111 on its own, clock tick by clock tick.

Small FSM

The IDLE → RUN → DONE controller from the previous slides, built from flip-flops and logic.

Together, these three pieces are the CPU's "memory-and-stepping fabric" — and, for this project, a complete automatic test rig for our decoder.

COMBINING BOTH WORLDS

Putting It Together

Sequential part

Clock + counter (or register) — remembers a value and steps it forward in time.

Combinational part

Our decoder — instantly turns whatever 4 bits it's given into one lit output.

Most real systems are exactly this mix: sequential elements that remember and step through time, driving combinational logic that reacts instantly — which is also how a full CPU works.

A PRACTICAL DETAIL

Resetting a Register: Sync vs Async

Every register or counter needs a known starting value — this is how it gets one.

SYNCHRONOUS RESET

The reset signal is only obeyed on the next clock edge — it waits its turn like any other input, which keeps every flip-flop changing in step.

Our counter would use this: on the IDLE → RUN transition, it's told "counter ← 0000" as an ordinary clocked update.

ASYNCHRONOUS RESET

The reset signal clears the flip-flop immediately, regardless of the clock — useful for forcing a known state the instant power turns on.

Faster to react, but it can ignore the clock entirely — which is why most classroom designs prefer the synchronous version.

STUDY AID

Module 3 Quick Reference

Latch

Memory cell that reacts whenever its control signal is high.

Flip-flop

Memory cell that reacts only at a clock edge.

Register

Several flip-flops storing one multi-bit value.

Clock

The steady signal that tells every flip-flop exactly when to update.

Counter

A register that adds 1 to itself every clock tick.

FSM

States + transitions describing step-by-step behaviour.

SUMMARY

Module 3 Takeaways

- Our converter is purely combinational — it has no memory, which is exactly what sets it apart from everything in this module.
- Latches and flip-flops are the basic memory cells; registers are just several flip-flops storing one multi-bit value, like our 4-bit input.
- A counter is a register that increments itself every clock tick — a natural way to automatically test all 16 inputs of our decoder.
- FSMs aren't needed for our one-step decoder, but they're exactly the tool — and a state table is exactly the format — for a real control unit later in the course.

Thank You

Design of a Four-Bit Binary to Decimal Conversion — Module 3 Basics